

02. Instalación y ejecución

PDF Reader v0.3.1 | Análisis 2026-09-24 | Commit e22b730

Contenido

- Requisitos
- Preparación
- Desarrollo Web
- Windows
- Android
- Validación
- Producción y despliegue
- Regenerar esta documentación PDF
- Errores frecuentes

Requisitos

Ámbito	Requisito
Todos	Git, Node.js 22 o superior, pnpm 11
Windows	Windows 10/11 para empaquetar y probar Electron
Android	JDK 21, Android SDK Platform 36 y Build Tools 36.0.0
PDF de documentación	Python con ReportLab y Poppler para render/QA

El repositorio exige pnpm: packageManager fija 11.19.0 y pnpm-lock.yaml es el único lockfile JavaScript.

Preparación

Bloque de código

```
git clone https://github.com/vladimiracunadev-create/pdf-reader-windows-android.git
cd pdf-reader-windows-android
pnpm install --frozen-lockfile
```

No hay .env ni variables obligatorias. PORT es opcional y vale 4173; la firma Android usa GitHub Secrets no versionados.

Desarrollo Web

Bloque de código

```
pnpm build:web
pnpm start:web
```

build:web crea dist/ con PDF.js y sus recursos; start:web lo sirve en http://localhost:4173. Para landing y demo:

Bloque de código

```
pnpm build:pages
pnpm start:pages
```

Windows

Bloque de código

```
pnpm start:desktop
pnpm build:windows
```

El primero abre Electron; el segundo genera NSIS y portable en `release/windows/`, sin Authenticode en 0.3.1.

Android

Bloque de código

```
pnpm android:init
pnpm android:assets
pnpm android:sync
pnpm android:debug
```

`android:init` crea el proyecto solo si falta. `android:sync` recompila Web, sincroniza Capacitor y reaplica versión/permisos. El APK debug queda en `android/app/build/outputs/apk/debug/app-debug.apk`.

Validación

Bloque de código

```
pnpm check
pnpm build:pages
pnpm release:check
```

`check` ejecuta 24 pruebas Node y `scripts/verify-repo.mjs`. `release:check` añade el build de Pages. El 2026-09-24, la instalación congelada y ambos gates finalizaron correctamente; CI repitió Web, Android y Windows sobre el commit etiquetado.

Producción y despliegue

La aplicación no mantiene un servidor de producción. GitHub Pages publica contenido estático. Los binarios se construyen en Actions cuando se empuja un tag `v*`; el tag debe coincidir con la versión y existir `docs/releases/vX.Y.Z.md`.

Regenerar esta documentación PDF

Con Python y ReportLab disponibles:

Bloque de código

```
pnpm docs:pdf
```

El script toma los 20 Markdown de `docs/system-documentation/` y reemplaza sus equivalentes en `docs/system-documentation/pdf/`. Poppler (`pdftopdf` y `pdftoppm`) se usa para la revisión visual, no para generar.

Errores frecuentes

- **Falta PDF.js:** ejecutar `pnpm install --frozen-lockfile`.
- `ERR_PNPM_ABORTED_REMOVE_MODULES_DIR_NO_TTY`: en CI definir `CI=true`; no borrar lockfiles.
- **SDK Android no encontrado:** configurar `ANDROID_HOME/ANDROID_SDK_ROOT` e instalar API 36.
- **Desincronización Android:** ejecutar `pnpm android:sync` y revisar `git diff -- android`.
- **Puerto ocupado:** definir otro PORT antes de `pnpm start:web`.
- **PDF cifrado:** el producto aún no solicita contraseñas.